

گزارش پیاده‌سازی زمان‌بند CFS و Fork COW در xv6

دلارام روحانی - ۴۰۲۳۱۰۲۶
امیرعباس انتظاری - ۴۰۲۳۱۰۰۶

۳ دی ۱۴۰۴

فهرست مطالب

۳	اول پیاده‌سازی زمان‌بند CFS
۳	۱ مقدمه به CFS
۳	۱.۱ مفاهیم کلیدی CFS
۳	۲ اهداف پیاده‌سازی CFS
۳	۳ جزئیات پیاده‌سازی CFS
۳	۱.۳ تغییرات در ساختار پردازش (proc.h)
۴	۲.۳ ساختار Min-Heap (minheap.c و minheap.h)
۴	۳.۳ متغیرهای سراسری CFS (proc.c)
۵	۴.۳ مقداردهی اولیه (procinit)
۵	۵.۳ تخصیص پردازش (allocproc)
۵	۶.۳ زمان‌بند جدید (scheduler)
۶	۷.۳ به‌روزرسانی (yield) و runtime
۶	۸.۳ اضافه کردن پردازش به صف (fork و wakeup)
۷	۴ فرمول محاسبه runtime
۷	۵ نتیجه‌گیری CFS
۸	دوم پیاده‌سازی Fork COW
۸	۶ مقدمه به COW
۸	۷ اهداف پیاده‌سازی COW
۸	۸ جزئیات پیاده‌سازی
۸	۱.۸ اضافه کردن بیت COW به PTE
۸	۲.۸ پیاده‌سازی شمارش ارجاع در kalloc.c
۹	۳.۸ پیاده‌سازی vm.c در uvmcopy_cow
۱۰	۴.۸ مدیریت Page Fault در trap.c

۵.۸	تابع cowfault	۱۰
۶.۸	سیستم کال cowfork	۱۱
۷.۸	سیستم کال physaddr	۱۱
۹	ثبت سیستم کال ها	۱۱
۱۰	برنامه تست	۱۱
۱۱	نتیجه گیری COW	۱۲

بخش اول

پیاده‌سازی زمان‌بند CFS

۱ مقدمه به CFS

Completely Fair Scheduler (CFS) یک الگوریتم زمان‌بندی است که در هسته لینوکس استفاده می‌شود. هدف اصلی CFS توزیع عادلانه زمان CPU بین تمام پردازنده‌ها است. در xv6 اصلی، از یک الگوریتم ساده Round-Robin استفاده می‌شود که هر پردازنده را به نوبت اجرا می‌کند. این روش ساده است اما عادلانه نیست زیرا اولویت پردازنده‌ها را در نظر نمی‌گیرد.

۱.۱ مفاهیم کلیدی CFS

- **vruntime (Virtual Runtime)**: زمان مجازی اجرای هر پردازنده. پردازنده‌ای که کمتر اجرا شده، vruntime کمتری دارد.
- **Weight**: وزن هر پردازنده بر اساس اولویت. پردازنده‌های مهم‌تر وزن بیشتری دارند.
- **Min-Heap**: برای انتخاب سریع پردازنده با کمترین vruntime از یک Min-Heap استفاده می‌شود.

۲ اهداف پیاده‌سازی CFS

- جایگزینی زمان‌بند Round-Robin با CFS
- پیاده‌سازی ساختار داده Min-Heap برای مدیریت صف اجرا
- اضافه کردن فیلدهای vruntime و weight به ساختار پردازنده
- به‌روزرسانی vruntime در هر بار yield

۳ جزئیات پیاده‌سازی CFS

۱.۳ تغییرات در ساختار پردازنده (proc.h)

دو فیلد جدید به ساختار proc اضافه شد:

```
1 struct proc {
2     // ... existing fields ...
3
4     int weight;           // Process weight (based on priority)
5     uint64 vruntime;      // Virtual runtime for CFS
6 };
```

Listing 1: struct proc در جدید فیلدهای

- **weight**: وزن پردازنده. مقدار پیش‌فرض ۱۰۲۴ (مطابق با nice=0 در لینوکس).
- **vruntime**: زمان مجازی اجرا. پردازنده با کمترین vruntime اولویت بالاتری برای اجرا دارد.

۲.۳ ساختار Min-Heap (minheap.h و minheap.c)

یک Min-Heap برای نگهداری پردازش‌های آماده اجرا پیاده‌سازی شد:

```
1 struct minheap {
2     struct proc *procs[NPROC]; // Array of process pointers
3     int size; // Current number of elements
4     int capacity; // Maximum capacity (NPROC)
5 };
```

Listing 2: ساختار Min-Heap

توابع اصلی:

- minheap_init(): مقداردهی اولیه heap
- minheap_insert(): اضافه کردن پردازش به heap
- minheap_extract_min(): استخراج پردازش با کمترین runtime

```
1 static void heapify_up(struct minheap *heap, int idx) {
2     while (idx > 0) {
3         int p = parent(idx);
4         if (heap->procs[idx]->vruntime < heap->procs[p]->vruntime) {
5             swap(heap, idx, p);
6             idx = p;
7         } else {
8             break;
9         }
10    }
11 }
```

Listing 3: تابع heapify_up

```
1 struct proc* minheap_extract_min(struct minheap *heap) {
2     if (heap->size == 0) return 0;
3
4     struct proc *min_proc = heap->procs[0];
5
6     // Move last element to root
7     heap->procs[0] = heap->procs[heap->size - 1];
8     heap->size--;
9
10    // Heapify down from root
11    if (heap->size > 0) {
12        heapify_down(heap, 0);
13    }
14
15    return min_proc;
16 }
```

Listing 4: تابع extract_min

۳.۳ متغیرهای سراسری (proc.c) CFS

در فایل proc.c متغیرهای سراسری زیر اضافه شد:

```

1 // CFS scheduler data structures
2 struct minheap run_queue; // Min-heap of runnable processes
3 struct spinlock runq_lock; // Lock for the run queue
4 uint64 min_vruntime = 0; // Global minimum vruntime

```

Listing 5: سراسری متغیرهای CFS

۴.۳ مقداردهی اولیه (procinit)

در تابع procinit() قفل و heap مقداردهی اولیه می‌شوند:

```

1 void procinit(void) {
2     initlock(&runq_lock, "runqueue");
3     minheap_init(&run_queue);
4     min_vruntime = 0;
5     // ...
6 }

```

Listing 6: CFS اولیه مقداردهی

۵.۳ تخصیص پردازنده (allocproc)

هنگام ایجاد پردازنده جدید، فیلدهای CFS مقداردهی می‌شوند:

```

1 static struct proc* allocproc(void) {
2     // ...
3
4     // Initialize CFS fields
5     p->vruntime = min_vruntime; // Start with current min
6     p->weight = 1024;           // Default weight (nice 0)
7
8     return p;
9 }

```

Listing 7: CFS فیلدهای مقداردهی در allocproc

۶.۳ زمان‌بند جدید (scheduler)

تابع scheduler() بازنویسی شد تا از CFS استفاده کند:

```

1 void scheduler(void) {
2     struct proc *p;
3     struct cpu *c = mycpu();
4     c->proc = 0;
5
6     for(;;){
7         intr_on();
8         intr_off();
9
10        // CFS: Extract process with minimum vruntime
11        acquire(&runq_lock);
12        p = minheap_extract_min(&run_queue);
13        release(&runq_lock);
14
15        if(p != 0) {
16            acquire(&p->lock);

```

```

17     if(p->state == RUNNABLE) {
18         p->state = RUNNING;
19         c->proc = p;
20         swtch(&c->context, &p->context);
21         c->proc = 0;
22     }
23     release(&p->lock);
24 } else {
25     asm volatile("wfi"); // Wait for interrupt
26 }
27 }
28 }

```

Listing 8: CFS زمان‌بند

۷.۳ به‌روزرسانی (yield) vruntime

در تابع yield()، vruntime پردازش به‌روز می‌شود:

```

1 void yield(void) {
2     struct proc *p = myproc();
3     acquire(&p->lock);
4
5     // CFS: Update vruntime based on weight
6     // vruntime += delta * (NICE_0_WEIGHT / weight)
7     p->vruntime += (1024 / p->weight);
8
9     // Update global min_vruntime
10    acquire(&runq_lock);
11    if(p->vruntime > min_vruntime)
12        min_vruntime = p->vruntime;
13    release(&runq_lock);
14
15    p->state = RUNNABLE;
16
17    // Add back to run queue
18    acquire(&runq_lock);
19    minheap_insert(&run_queue, p);
20    release(&runq_lock);
21
22    sched();
23    release(&p->lock);
24 }

```

Listing 9: yield در vruntime به‌روزرسانی

۸.۳ اضافه کردن پردازش به صف (fork و wakeup)

در توابع fork() و wakeup()، پردازش‌ها به run_queue اضافه می‌شوند:

```

1 // In fork:
2 np->state = RUNNABLE;
3 acquire(&runq_lock);
4 minheap_insert(&run_queue, np);
5 release(&runq_lock);

```

Listing 10: اجرا صف به‌اضافه

۴ فرمول محاسبه $vruntime$

فرمول به‌روزرسانی $vruntime$ به صورت زیر است:

$$vruntime_{new} = vruntime_{old} + \frac{NICE_0_WEIGHT}{weight} \times \Delta t \quad (۱)$$

که در آن:

• $NICE_0_WEIGHT = 1024$: وزن پیش‌فرض

• $weight$: وزن پردازش فعلی

• Δt : زمان اجرا (در پیاده‌سازی ساده برابر ۱)

۵ نتیجه‌گیری CFS

با پیاده‌سازی CFS:

- پردازش‌هایی که کمتر اجرا شده‌اند، اولویت بالاتری می‌گیرند
- پردازش‌های با وزن بالاتر، $vruntime$ کمتری افزایش می‌یابد (زمان بیشتری می‌گیرند)
- انتخاب پردازش بعدی با پیچیدگی $O(\log n)$ انجام می‌شود

بخش دوم

پیاده‌سازی COW Fork

۶ مقدمه به COW

در سیستم عامل، زمانی که فراخوانی سیستمی `fork()` اجرا می‌شود، یک کپی کامل از فضای آدرس پردازنده والد برای پردازنده فرزند ایجاد می‌شود. این روش سنتی باعث اتلاف حافظه و زمان می‌شود، به خصوص زمانی که فرزند بلافاصله `exec()` را فراخوانی کند.

Copy-on-Write (COW) یک تکنیک بهینه‌سازی است که در آن صفحات حافظه بین والد و فرزند به اشتراک گذاشته می‌شوند تا زمانی که یکی از آن‌ها بخواهد بنویسد. در آن لحظه، یک کپی از صفحه ایجاد می‌شود.

۷ اهداف پیاده‌سازی COW

- پیاده‌سازی `cowfork()` که صفحات را به جای کپی کردن، به اشتراک می‌گذارد
- مدیریت `Page Fault` برای تشخیص نوشتن روی صفحات COW
- پیاده‌سازی شمارش ارجاع (`Reference Counting`) برای صفحات فیزیکی
- اضافه کردن سیستم‌کال `physaddr()` برای تست صحت پیاده‌سازی

۸ جزئیات پیاده‌سازی

۱.۸ اضافه کردن بیت COW به PTE

برای تشخیص صفحات COW از صفحات فقط‌خواندنی معمولی، یک بیت جدید به فلگ‌های PTE اضافه کردیم. از بیت `RSW (Reserved for Software)` استفاده شد:

```
1 #define PTE_V (1L << 0) // valid
2 #define PTE_R (1L << 1)
3 #define PTE_W (1L << 2)
4 #define PTE_X (1L << 3)
5 #define PTE_U (1L << 4) // user can access
6 #define PTE_COW (1L << 8) // copy-on-write page (RSW bit)
```

Listing 11: بیت تعریف COW در `riscv.h`

۲.۸ پیاده‌سازی شمارش ارجاع در `kalloc.c`

برای مدیریت صحیح حافظه مشترک، یک آرایه برای نگهداری تعداد ارجاعات هر صفحه فیزیکی اضافه شد:

```
1 #define MAXPAGES 32768
2 #define PA2IDX(pa) (((uint64)(pa) - KERNBASE) / PGSIZE)
3
4 struct {
5     struct spinlock lock;
6     int count[MAXPAGES];
7 } pageref;
```

Listing 12: ارجاع شمارش ساختار

سه تابع برای مدیریت ارجاعات اضافه شد:

- `kref_incr(pa)`: افزایش شمارنده ارجاع
- `kref_decr(pa)`: کاهش شمارنده ارجاع
- `kref_get(pa)`: خواندن مقدار شمارنده

همچنین تابع `kfree()` اصلاح شد تا فقط زمانی صفحه را آزاد کند که شمارنده به صفر برسد:

```
1 void kfree(void *pa) {
2     // Only free if reference count reaches 0
3     acquire(&pageref.lock);
4     if(pageref.count[PA2IDX(pa)] > 1) {
5         pageref.count[PA2IDX(pa)]--;
6         release(&pageref.lock);
7         return; // Don't free, still referenced
8     }
9     pageref.count[PA2IDX(pa)] = 0;
10    release(&pageref.lock);
11    // ... rest of kfree
12 }
```

Listing 13: اصلاح شده `kfree` تابع

۳.۸ پیاده سازی `uvmcopy_cow` در `vm.c`

این تابع جایگزین `uvmcopy` در `cowfork` می شود. به جای کپی کردن صفحات:

۱. صفحات را فقط خواندنی می کند

۲. بیت COW را ست می کند

۳. هر دو جدول صفحه (والد و فرزند) را به یک صفحه فیزیکی نگاشت می کند

۴. شمارنده ارجاع را افزایش می دهد

```
1 int uvmcopy_cow(pagetable_t old, pagetable_t new, uint64 sz) {
2     for(i = 0; i < sz; i += PGSIZE){
3         // ... get PTE and physical address
4
5         // If writable, mark as COW and remove write permission
6         if(flags & PTE_W) {
7             flags = (flags & ~PTE_W) | PTE_COW;
8             *pte = PA2PTE(pa) | flags | PTE_V; // Update parent
9         }
10
11        // Map same physical page in child
12        mappages(new, i, PGSIZE, pa, flags);
13
14        // Increment reference count
15        kref_incr((void*)pa);
16    }
17    return 0;
18 }
```

Listing 14: `uvmcopy_cow` تابع

۴.۸ مدیریت Page Fault در trap.c

وقتی پردازش روی صفحه COW می‌نویسد، یک Store Page Fault با `scause=15` رخ می‌دهد. در `usertrap()` این را مدیریت می‌کنیم:

```
1 } else if(r_scause() == 15) {
2     uint64 va = r_stval();
3     pte_t *pte = walk(p->pagetable, va, 0);
4
5     // Check if it's a COW page fault
6     if(pte != 0 && (*pte & PTE_V) && (*pte & PTE_COW)) {
7         if(cowfault(p->pagetable, va) < 0) {
8             setkilled(p);
9         }
10    } else if(vmfault(p->pagetable, va, 0) == 0) {
11        setkilled(p);
12    }
13 }
```

Listing 15: مدیریت Page Fault در usertrap

۵.۸ تابع cowfault

این تابع مسئول رسیدگی به خطای COW است:

```
1 int cowfault(pagetable_t pagetable, uint64 va) {
2     va = PGROUNDDOWN(va);
3     pte_t *pte = walk(pagetable, va, 0);
4
5     // Check if COW page
6     if((*pte & PTE_COW) == 0) return -1;
7
8     uint64 pa = PTE2PA(*pte);
9
10    // If only one reference, just restore write permission
11    if(kref_get((void*)pa) == 1) {
12        *pte = (*pte | PTE_W) & ~PTE_COW;
13        return 0;
14    }
15
16    // Allocate new page and copy content
17    char *mem = kalloc();
18    memmove(mem, (char*)pa, PGSIZE);
19
20    // Update PTE with new page, restore write, clear COW
21    uint flags = (PTE_FLAGS(*pte) | PTE_W) & ~PTE_COW;
22    *pte = PA2PTE(mem) | flags | PTE_V;
23
24    // Decrement reference on old page
25    kfree((void*)pa);
26
27    return 0;
28 }
```

Listing 16: تابع cowfault

۶.۸ سیستم‌کال `cowfork`

تابع `kcowfork()` در `proc.c` مشابه `kfork()` است، با این تفاوت که از `uvmcopy_cow()` استفاده می‌کند:

```
1 int kcowfork(void) {
2     // Allocate process
3     if((np = allocproc()) == 0) return -1;
4
5     // Share user memory using COW (not copy!)
6     if(uvmcopy_cow(p->pagetable, np->pagetable, p->sz) < 0){
7         freeproc(np);
8         return -1;
9     }
10    // ... rest same as kfork
11 }
```

Listing 17: تابع `kcowfork`

۷.۸ سیستم‌کال `physaddr`

برای تست صحت پیاده‌سازی، این سیستم‌کال شماره صفحه فیزیکی یک آدرس مجازی را برمی‌گرداند:

```
1 uint64 sys_physaddr(void) {
2     struct proc *p = myproc();
3     uint64 va;
4     argaddr(0, &va);
5
6     va = PGROUNDDOWN(va);
7     pte_t *pte = walk(p->pagetable, va, 0);
8     uint64 pa = PTE2PA(*pte);
9
10    return pa / PGSIZE; // Return page number
11 }
```

Listing 18: سیستم‌کال `physaddr`

۹ ثبت سیستم‌کال‌ها

دو سیستم‌کال جدید به فایل‌های زیر اضافه شد:

- `syscall.h`: تعریف `SYS_cowfork=24` و `SYS_physaddr=25`

- `syscall.c`: ثبت توابع `handler`

- `usys.pl`: تولید‌های `stub` سمت کاربر

- `user.h`: اعلان توابع برای برنامه‌های کاربر

۱۰ برنامه تست

برنامه `cowtest.c` صحت پیاده‌سازی را بررسی می‌کند:

```

1 int main() {
2     char *buf = malloc(4096);
3     buf[0] = 'A';
4
5     int parent_page_before = physaddr(buf);
6
7     if(cowfork() == 0) {
8         // Child
9         int child_page_before = physaddr(buf);
10        buf[0] = 'C'; // Triggers COW!
11        int child_page_after = physaddr(buf);
12
13        // child_page_before == child_page_after means COW failed
14        // child_page_before != child_page_after means COW worked
15    }
16
17    wait(0);
18    // Parent's buf[0] should still be 'A'
19 }

```

Listing 19: تست برنامۀ COW

۱۱ نتیجه‌گیری COW

پیاده‌سازی COW Fork با موفقیت انجام شد. خروجی مورد انتظار:

- قبل از نوشتن: شماره صفحه والد و فرزند یکسان است (اشتراک‌گذاری)
- بعد از نوشتن توسط فرزند: شماره صفحه فرزند تغییر می‌کند (کپی شده)
- داده والد تغییر نمی‌کند (ایزوله‌سازی صحیح)

این پیاده‌سازی باعث صرفه‌جویی قابل توجه در حافظه و زمان می‌شود، به‌خصوص در سناریوهای fork+exec.